LABORATORIUM 4

Systemy z wiedzą niepewną: logika rozmyta

Przedmiot	Sztuczna inteligencja
Forma	Laboratorium
Czas	4 h dydaktyczne (190 min), 01.12.2026
Grupy	INLS5_SI
Prowadzący	mgr inż. Szymon Guzik

Cele zajęć

- Odróżnić niepewność probabilistyczną od nieprecyzyjności pojęć („ciepło”, „szybko”).
- Zdefiniować zbiory rozmyte i funkcje przynależności.
- Zbudować sterownik rozmyty Mamdaniego — fuzyfikacja, reguły, agregacja, defuzyfikacja.
- Dostroić reguły i funkcje przynależności na podstawie zachowania sterownika.
- Zintegrować moduł z projektem (etap 3).



Logika rozmyta — „trochę ciepło” zamiast „ciepło albo zimno”

1. Plan zajęć (190 minut)

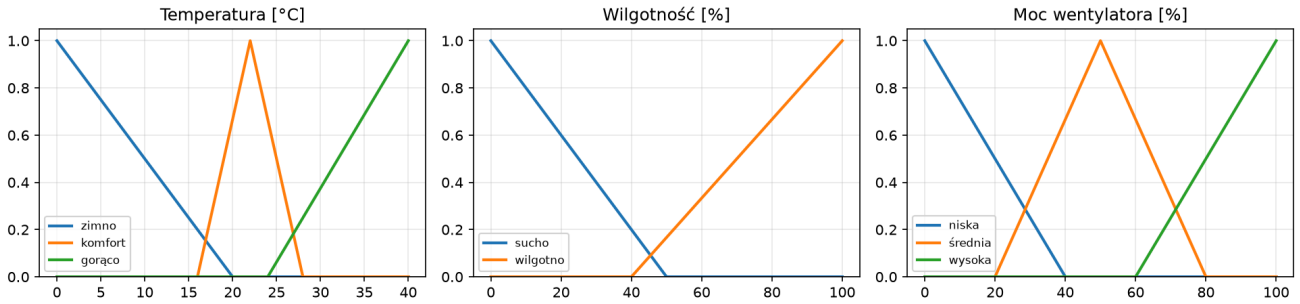
Czas	Co robimy
0–25 min	Niepewność i nieprecyzyjność, zbiory rozmyte, funkcje przynależności
25–75 min	Sterownik Mamdaniego — przykład, zadania 1–2
75–85 min	Przerwa
85–140 min	Własny sterownik — zadanie 3, porównanie z regułami ostrymi
140–190 min	Projekt — etap 3: integracja modułów, konsultacje

2. Teoria

Nieprecyzyjność zamiast przypadku

Rachunek prawdopodobieństwa opisuje **niepewność co do zdarzenia** („czy jutro spadnie deszcz?”). Logika rozmyta (Lotfi Zadeh, 1965) opisuje **nieostre pojęcia**: 23 °C nie jest ani w pełni „ciepło”, ani w pełni „komfortowo”. W klasycznym zbiorze element należy albo nie należy (0 lub 1). W zbiorze rozmytym ma **stopień przynależności** $\mu \in [0, 1]$. Dzięki temu reguły w rodzaju „jeżeli jest gorąco i wilgotno, włącz mocno wentylator” działają płynnie, bez skokowych przełączeń.

Pojęcie	Znaczenie
Zmienna lingwistyczna	Np. „temperatura” z wartościami: zimno, komfort, gorąco
Funkcja przynależności $\mu(x)$	Stopień, w jakim x należy do zbioru (trójkątna, trapezowa, gaussowska)
T-norma (AND)	Najczęściej minimum: $\mu(A \wedge B) = \min(\mu A, \mu B)$
S-norma (OR)	Najczęściej maksimum: $\mu(A \vee B) = \max(\mu A, \mu B)$
Negacja	$\mu(\neg A) = 1 - \mu A$
Defuzyfikacja	Zamiana wyniku rozmytego na liczbę — np. środek ciężkości (centroid)



Funkcje przynależności sterownika z przykładu (wykres wygenerowany przez program)

3. Sterownik Mamdaniego



Cztery kroki wnioskowania rozmytego Mamdaniego

1. **Fuzyfikacja** — dla zmierzonej temperatury i wilgotności liczymy stopnie przynależności do każdego zbioru.
2. **Wnioskowanie** — dla każdej reguły stopień odpalenia to minimum stopni przesłanek; wniosek (zbiór wyjściowy) zostaje „przycięty” na tej wysokości.
3. **Agregacja** — przycięte zbiory wyjściowe łączymy maksimum.
4. **Defuzyfikacja** — liczymy środek ciężkości powstałej figury: $\Sigma\, x \cdot \mu(x) \,/\, \Sigma\, \mu(x)$.

Przykład — wentylator

Pięć reguł steruje mocą wentylatora. Dla 27 °C i 70% wilgotności odpalają dwie reguły naraz („komfort i wilgotno → średnia” oraz „gorąco i wilgotno → wysoka”), a wynik 62,5% leży pomiędzy — to właśnie płynność, której nie da sterownik z regułami ostrymi.

sterownik_rozmyty.py

```python
"""Sterownik rozmyty Mamdaniego: temperatura i wilgotność -> moc wentylatora [%] (defuzyfikacja środkiem ciężkości)."""
import sys


def trojkat(x, a, b, c):
    """Trójkątna funkcja przynależności (a, c — podstawa, b — wierzchołek); a == b lub b == c daje bark."""
    if x < a or x > c:
        return 0.0
    if x == b:
        return 1.0
    return (x - a) / (b - a) if x < b else (c - x) / (c - b)


TEMP = {"zimno": (0, 0, 20), "komfort": (16, 22, 28), "gorąco": (24, 40, 40)}          # °C
WILG = {"sucho": (0, 0, 50), "wilgotno": (40, 100, 100)}                               # %
MOC = {"niska": (0, 0, 40), "średnia": (20, 50, 80), "wysoka": (60, 100, 100)}         # %

REGULY = [  # (temperatura, wilgotność lub None = dowolna, moc)
    ("zimno", None, "niska"),
    ("komfort", "sucho", "niska"),
    ("komfort", "wilgotno", "średnia"),
    ("gorąco", "sucho", "średnia"),
    ("gorąco", "wilgotno", "wysoka"),
]


def steruj(t, w, slad=False):
    mt = {k: trojkat(t, *v) for k, v in TEMP.items()}          # 1. fuzyfikacja
    mw = {k: trojkat(w, *v) for k, v in WILG.items()}
    aktywacja = {k: 0.0 for k in MOC}
    for rt, rw, wy in REGULY:                                   # 2. wnioskowanie (AND = min, łączenie = max)
        stopien = min(mt[rt], mw[rw]) if rw else mt[rt]
        aktywacja[wy] = max(aktywacja[wy], stopien)
        if slad and stopien > 0:
            print(f"   JEŻELI temp={rt:8} I wilg={rw or '*':8} TO moc={wy:8} stopień {stopien:.2f}")
    xs = [i * 0.5 for i in range(201)]                          # 3. agregacja + 4. defuzyfikacja (centroid)
    mu = [max(min(aktywacja[k], trojkat(x, *MOC[k])) for k in MOC) for x in xs]
    return sum(x * m for x, m in zip(xs, mu)) / sum(mu) if sum(mu) else 0.0


def wykres(plik):
    import matplotlib.pyplot as plt
    fig, osie = plt.subplots(1, 3, figsize=(13, 3.2))
    for os_, (nazwa, zbiory, zakres) in zip(osie, [("Temperatura [°C]", TEMP, 40), ("Wilgotność [%]", WILG, 100),
                                                   ("Moc wentylatora [%]", MOC, 100)]):
        xs = [zakres * i / 400 for i in range(401)]
        for etykieta, p in zbiory.items():
            os_.plot(xs, [trojkat(x, *p) for x in xs], label=etykieta, linewidth=2)
        os_.set_title(nazwa); os_.set_ylim(0, 1.05); os_.legend(fontsize=8); os_.grid(alpha=.3)
    fig.tight_layout(); fig.savefig(plik, dpi=150)


if __name__ == "__main__":
    for t, w in [(10, 60), (23, 30), (27, 70), (35, 85)]:
        print(f"temperatura {t} °C, wilgotność {w} %:")
        print(f"   => moc wentylatora {steruj(t, w, slad=True):.1f} %")
    if len(sys.argv) > 1:
        wykres(sys.argv[1])

```

kod/sterownik_rozmyty.py

```
$ python3 -B kod/sterownik_rozmyty.py wykres_przynaleznosci.png
temperatura 10 °C, wilgotność 60 %:
   JEŻELI temp=zimno    I wilg=*       TO moc=niska    stopień 0.50
   => moc wentylatora 15.4 %
temperatura 23 °C, wilgotność 30 %:
   JEŻELI temp=komfort  I wilg=sucho    TO moc=niska    stopień 0.40
   => moc wentylatora 16.2 %
temperatura 27 °C, wilgotność 70 %:
```

```
   JEŻELI temp=komfort  I wilg=wilgotno TO moc=średnia  stopień 0.17
   JEŻELI temp=gorąco   I wilg=wilgotno TO moc=wysoka   stopień 0.19
   => moc wentylatora 62.5 %
temperatura 35 °C, wilgotność 85 %:
   JEŻELI temp=gorąco   I wilg=wilgotno TO moc=wysoka   stopień 0.69
   => moc wentylatora 85.8 %
```

Zadanie 1 (20 min)

Wygeneruj **powierzchnię sterowania**: policz moc dla temperatur 0–40 °C (co 1) i wilgotności 0–100% (co 5) i narysuj ją jako mapę ciepła (`matplotlib.pyplot.imshow`). Gdzie są „schodki” i skąd się biorą?

Zadanie 2 (20 min)

Zamień trójkątne funkcje przynależności na **trapezowe** (dodaj funkcję `trapez(x, a, b, c, d)`) tak, żeby „komfort” miał płaski wierzchołek 20–24 °C. Jak zmieniła się powierzchnia sterowania?

Zadanie 3 (45 min)

Zaprojektuj własny sterownik rozmyty (wybierz jeden): a) **tempomat**: odległość od auta z przodu + różnica prędkości → hamowanie/przyspieszenie, b) **napiwek**: jakość obsługi + jakość jedzenia → napiwek w %, c) **podlewanie**: wilgotność gleby + prognoza deszczu → czas podlewania. Minimum: 2 wejścia po 3 zbiory, 1 wyjście, 7 reguł. Porównaj z wersją „ostrą” (zwykłe `if`) na 5 przypadkach.

Biblioteki

W projektach możecie użyć gotowej biblioteki `scikit-fuzzy` (`pip install scikit-fuzzy`). Na zajęciach piszemy sterownik ręcznie, żeby zobaczyć każdy krok obliczeń.

4. Projekt — etap 3

Integracja (50 min)

Zespoły łączą moduły w jeden program z prostym interfejsem (konsola, Streamlit lub Flask), piszą testy dla 2–3 kluczowych funkcji i uzupełniają `README.md` o instrukcję uruchomienia. Prowadzący przegląda repozytoria.

5. Literatura

- D. Rutkowska, M. Piliński, L. Rutkowski, *Sieci neuronowe, algorytmy genetyczne i systemy rozmyte*, PWN 1997.
- M. Flasiński, *Wstęp do sztucznej inteligencji*, PWN 2011 — rozdz. o logice rozmytej.
- L. A. Zadeh, „Fuzzy sets”, *Information and Control*, 1965.